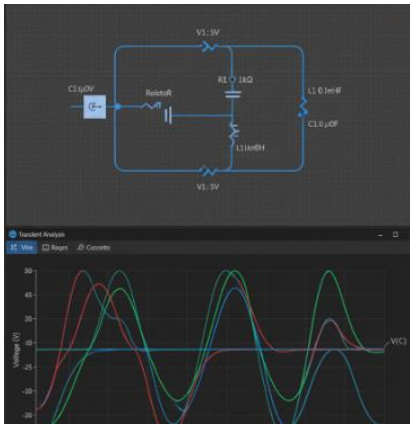
University of Southampton

ELEC Laboratory Programme

Laboratory manual

ρ2

Circuit Simulator



The goal of this project is to design a simple circuit simulator. You are expected to apply principles of Object Oriented Programming in completing this project. This lab is heavily inspired by the C labs in part 1 ELEC1312.

Revision History

October 7, 2025	Dr Ivan Ling	New for 2025/26
October 1, 2026	Dr Ivan Ling	Revised content

Electronics and Computer Science

© Electronics and Computer Science, University of Southampton

Laboratory Overview and Information

Schedule	
Preparation time:	3 hour
Lab time:	3 hours
Items provided	
Tools:	None
Components:	None
Equipment:	None
Software:	G++ Compiler
Items to bring	
Essentials:	A full list is available on the Laboratory website at https://secure.ecs.soton.ac.uk/notes/ellabs/databook/essentials/

Initial Instructions

Before you come to the lab, it is essential that you read through this document and complete ***all*** of the preparation work in section 2. Then you may proceed to complete the rest of the lab to get marked during the session. Alternatively, you could spend the time in the lab to complete them.

Academic Integrity

If you undertake the preparation jointly with other students, it is important that you acknowledge this fact in your logbook. Similarly, you may want to use sources from the internet or books to help answer some of the questions. Again, record any sources in your logbook.

© Electronics and Computer Science, University of Southampton

1 Aims, Learning Outcomes and Outline

This laboratory exercise aims to:

- Get your hands dirty with a practical implementation of object-oriented programming.
- Level up your understanding of the Software Development Lifecycle.
- Build a multi-feature software system from the ground up.

Having successfully completed the lab, you will be able to:

- Understand classes and how they work
- Use abstraction and encapsulation effectively
- Determine the best data structures to use based on given problems

Congratulations on making it this far and surviving the Sudoku lab. Just a reminder, you will have the opportunity to work on this lab throughout the week and finally in week 4, you will be showcasing your results during the lab session and be marked by the lab demonstrators. You will also have opportunities to get some help from the demonstrators during the lab sessions (but expect your scores to be slightly lower if help is required).

The circuit simulator lab is heavily inspired by the C labs in ELEC1312 last year. Consider this the long-awaited sequel… Perhaps this is a good time to dig out that old logbook!

The central goal of this lab is the development of a simple circuit simulator. Granted, this simulator will have a very limited real-world use aside from being a good lab exercise.

This project requires you to engineer a software application with a range of user-selectable features. Your responsibilities will include not only implementing these functions but also designing a clear and intuitive Command-Line Interface (CLI) that allows users to navigate and select the various simulation options.

Lab Partners 👫
For this assignment, you may choose to collaborate with your project partner. However, it is important to understand that all students will be assessed individually.

If you work in a pair, we highly recommend a classic divide-and-conquer strategy. This is a practical opportunity to apply the team-based software development methods covered in lectures, so don't be a lone wolf!

© Electronics and Computer Science, University of Southampton

2 Preparation – Refresh your memory!

In this lab, you will be required to build a **command line interface (CLI)** multifunctional controlled circuit simulator.

2.1 Graph plotter

Any good simulator needs a good graph plotting function to visualize the simulation output.



Figure 1: Sinewave plotter from C2 Lab

2.1.1 Create required class

Read through your old codes from C2 and work out which elements should be converted into classes and implemented. Remember! The key is to make the codes as reusable as possible!

Document 📝 this in your logbook as UML class diagrams.

Convert the codes to C++ following your UML class diagrams. (A very annoyingly bad version of the c2 answer is provided if you didn't have your old C2 codes with you…)

2.1.2 Implement Fourier Series

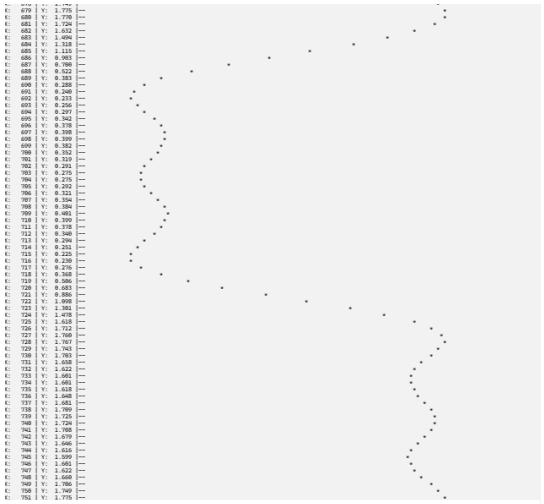


Figure 2: Fourier series on terminal

Read through the Optional Additional work of the C2 lab. Try to create a Fourier Series simulator that allows user to enter up to 6 Fourier Series Coefficients and generate the corresponding graph using the library you've created (**Bonus:** Implement both sin and cos in the series). Input a few known coefficients (e.g., for a sawtooth wave) and confirm the output graph looks as expected. Check this link for an interactive online version: https://www.desmos.com/calculator/lab9nylxsi

Hint: It might be helpful to package all the classes, functions and definitions as a library with separate .h and .cpp files to be used in the lab exercise.

© Electronics and Computer Science, University of Southampton

3 Building a simple simulator

To understand how classes interact with one another, we will build a numerical simulator for a physical system. A classic example of a system governed by a first-order differential equation is a Resistor-Capacitor (RC) circuit.

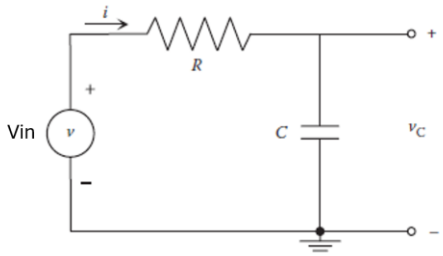


Figure 3: Simple RC circuit

When a voltage V_{in} is applied to an RC circuit, the voltage across the capacitor V_c changes over time according to this differential equation:

$$\frac{dV_c}{dt} = \frac{V_{in} - V_c}{R \cdot C}$$

Instead of solving this analytically, our simulator will solve it numerically using Euler's Method. We can approximate the next voltage state at a small timestep, Δt as:

$$V_c(t + \Delta t) = V_c(t) + \Delta t \cdot \left(\frac{V_{in} - V_c(t)}{R \cdot C}\right)$$

Essentially, what we're showing is that the capacitor voltage at the next timestep is equal to the capacitor voltage at the current time step plus an update term ($\Delta t \cdot \frac{dV_c}{dt}$).

3.1 Circuit class

Before writing any code, we must design our class structure. A good object-oriented design hides the complex math inside the class and exposes simple ways for the user to interact with it.

- **Identify Attributes (Private member variables):** What physical properties does an RC circuit have? (Hint: Resistance R, Capacitance C, and its current state V_c).
- **Identify Methods (Public member functions):** What do we need to do with this circuit? We need to initialize it (Constructor), set the input voltage, step the simulation forward by Δt, and read the current capacitor voltage.

Document 📝 your proposed RCCircuit UML class diagram in your logbook. Ensure you clearly indicate which members are public (+) and which are private (-).

❓ Hint: If you need some help here, look for rc_circuit.h in the HINTs folder.

© Electronics and Computer Science, University of Southampton

- Test each of your three modules in isolation. Create a simple main() function for each one that calls its core features.
- Create simple, predictable test cases.
- Verify your outputs

Document 📝 your testing process and results in your logbook. This step will save you from a world of pain later.

3.2 GUI Integration

Now, bring together Part 1 and Part 2. In your main.cpp:

1. Instantiate your RCCircuit object with $R = 1000\Omega$ and $C = 0.001F$.
2. Create a simulation loop that runs for 5 seconds of simulation time, with a time step of $\Delta t = 0.01s$.
3. Apply a constant step input of $V_{in} = 5V$.
4. Store the V_c results in an array or vector and pass this data to your **Graph Plotter** class from Section 1 to visualize the capacitor charging curve.

❓ Hint: Stuck here? Download the main.cpp from the HINTs folder.

3.3 Simple Control Circuit Simulation

Now that we have a working physical simulator, we will create a second class: a Controller. This demonstrates how two different objects interact in C++.

We want the capacitor to reach and maintain a specific target voltage (e.g., 3V), but we can only toggle the power supply fully ON (5V) or fully OFF (0V). This requires a simple "Bang-Bang" controller.

Design and implement a Controller class:

- **Attributes:** Target Voltage, Maximum Output Voltage, Minimum Output Voltage.
- **Methods:** A double calculateOutput(double current_voltage) function. If the current voltage is lower than the target, return Maximum Output. If it is higher, return Minimum Output.

Modify your simulation loop in main.cpp to use both objects. For each time step Δt:

1. Read the current voltage from the RCCircuit object.
2. Pass this voltage to the Controller object's calculateOutput method.
3. Feed the resulting output voltage back into the RCCircuit object's step method.
4. Plot the result.
5. You should now see a graph where the voltage rapidly climbs and then oscillates tightly around your target voltage!

Record 📝 your observations!

© Electronics and Computer Science, University of Southampton

4 Optional Additional Work

⚠ This section is an extension task for those who have already completed the main project.

Try implementing one of the following to push your OOP skills further:

- **Proportional (P) Controller Class:** Instead of Bang-Bang control, create a new class called PController that outputs a voltage proportional to the error: $V_{out} = K_p \cdot (V_{target} - V_c)$. How does the graph change? Can you vary the

- **RLC Circuit (2nd Order ODE):** Create an RLCCircuit class. This will require two state variables (Current I and Voltage V_c) and solving two coupled differential equations simultaneously in your step function. What can you observe?

- **Better UI/UX:** You must have noticed now that the main deliverables for this lab are the libraries. Being part of a dev team means that your library must be easily understandable by your teammates.

  However, building an impressive UI/UX to showcase all the capabilities of your library could put you on another level. Warning though! Please spend sensible amount of time on this… It's very easy to obsess over the UI/UX and spend an insane amount of time on this…

© Electronics and Computer Science, University of Southampton